Read Everywhere, Verify There: What It Takes to Put a Frozen-Model Read-Out on the Visitor's Hardware

James Burton Lancaster

August 24, 2026

Abstract

A companion paper established that the linearly readable structure in a frozen language model is invariant along every axis a deployment can vary, and that a head trained once transfers across model scale, weight precision, and silicon with at most a small recalibration. That result makes browser- and edge-tier deployment *possible*. This paper reports what happens when you actually build it, and finds that possibility and correctness are separated by three failures that only an end task can see.

We ship a single 0.6B-parameter quantized language model into a web browser where it does two jobs from one forward pass: it answers the visitor, and the hidden state that answer was computed from is read out to search 123,287 photographs that a 27B-parameter model encoded offline. The large model is never downloaded and never runs.

Three findings. First, **the recalibration is load-bearing and its absence is silent**: moving a head from PyTorch/fp16 to candle/Q4_0 in WebAssembly drops text-to-image R@1 from 0.230 to 0.0154, a fifteen-fold collapse that a 4 KB mean vector restores to 0.195, and which no agreement metric can detect because agreement applies the same transform to both sides of the comparison. Second, **retrieval quality reported on small pools does not survive deployment scale**: the same head and captions score R@1 0.2306 against 1,000 images and 0.0148 against the 123,287 actually searched. Third, **steering a read-out must be calibrated on retention, not on effect size**: class purity rises monotonically to 0.9 while the query's own neighbourhood collapses to 0.01, so optimising the visible effect optimises the query away.

We report the payload accounting that makes the tier real (int8 index storage is free: R@1 0.2306 against 0.2300 at a quarter the resident memory), an ablation in which four of six registered predictions were wrong, and a negative-results ledger in which the intervention we expected to be free and helpful was the most harmful thing we tried.

1. Introduction

The companion paper to this one, *Train Once, Read Everywhere*, established that the structure a small linear head reads out of a frozen language model is a property of the model class rather than of any particular host. A head trained on one substrate reads on a host ten times smaller, on the same host at 4-bit precision, and on entirely different silicon and kernels, with no retraining and at most a 42 KB recalibration. Deployment tiers, it concluded, differ in latency and cost, never in capability.

That claim is about what is *possible*. This paper is about what it costs to make it *true* for a specific visitor on a specific device, and the honest answer is that the gap is larger and stranger than the invariance result alone suggests. Invariance says the information survives the journey. It does not say your instrument is still pointed at it when it arrives.

We built the browser tier and measured it. A visitor loads a page; a 0.6B quantized model downloads once and stays cached; from then on the visitor can converse with it, and every message they send also searches a gallery of 123,287 photographs, because the vector that searches is a read-out on a hidden state of the same frozen model. There is no inference server. Nothing the visitor types leaves the tab.

The three findings we report are all of the same character: each is a way the system produces confident, plausible, wrong output, and each is invisible to the metric one would naturally reach for.

Contributions.

1. A measurement of how load-bearing the per-runtime recalibration is on an end task, and a demonstration that the standard portability check — agreement between runtimes in head space — is structurally blind to its absence (§3).
2. Retrieval measured at deployment scale rather than benchmark scale, with the cost of the difference reported explicitly (§4).
3. A calibration procedure for read-out steering that uses neighbourhood retention rather than effect size, with matched random controls (§5).
4. Payload accounting for the tier, including the result that int8 index storage is free (§6).
5. An ablation with predictions registered before running, of which four of six were wrong, and a negative-results ledger (§7, §8).

2. The deployment

2.1 One model, two jobs

The usual way to put conversation and search on one page is two models: a language model for the dialogue and an embedding model for the index. We ship one. The tokens the visitor reads and the vector that searches the gallery come out of the same frozen weights, because the read-out head consumes a hidden state of that model rather than an embedding from another.

This is not an optimisation. It is the program's claim made operational: if retrieval is a read-out on the representation rather than a separate skill, then a model that can talk can also search. In the deployed system the retrieved images are posted to the page *before* the sentence is, because the read that produces them finishes long before generation does.

We are precise about what is shared, because the tidier claim is false. The deployment reads the message twice through the same weights: once bare for the read-out, and once inside the chat template for generation. It does not reuse generation's prefill, and we measured what doing so would cost. Handing the head the templated prompt instead of the bare message moves the median rank of the correct photograph from 187 to 41,014 out of 123,287, and the two projected vectors have a cosine of 0.10. The head is fitted on one distribution of hidden states and chat formatting is a different one, which is the lesson of §3 arriving by another route. A design wanting one shared prefill must fit the head on templated states or carry a correction between the two.

2.2 An asymmetry that shapes everything

The two sides of the bridge have completely different economics, and recognising this changes what is worth optimising.

The **text head** ships. Every parameter is a byte the visitor downloads.

The **image head** does not. It runs offline on a datacenter GPU; the browser receives only the *projected gallery*, whose size is fixed by the index format regardless of how the projection was computed. Depth on the image side is free in payload terms.

We exploited this asymmetry and it backfired, which is §7.

2.3 What ships

artifact	size	notes
quantized text model (GGUF Q4_0)	382 MB	fetched once, cached
WebAssembly engine	5.7 MB	model loading, tap, projection, search
read-out head (text side, fp16)	2.1 MB	image side never ships
gallery index (123,287 × 1024, int8)	130 MB	§6
runtime anchor	4 KB	§3
thumbnail mirror	484 MB + 1 MB	byte-range addressed, §6.2

The 27B-parameter image tower appears nowhere in this table. It encoded the gallery months earlier and left behind a file. A 0.6B model on a phone meets it in that space.

2.4 The read-out geometry, ported

The deployed arithmetic is four operations: centre a hidden state by its modality anchor, project it, L2-normalise, and search by dot product. We implemented these in Rust with no model dependency, compiling to both `wasm32-unknown-unknown` and native targets, and pinned the port to the Python path that produced every published number in the program.

Verification is at two levels. A fixture of real hidden states with their expected projections checks the projection itself. A replay fixture checks the whole deployment path — state, head, index, ranked results — on real SugarCrepe images and COCO captions. The Rust implementation reproduces the Python ranking exactly across 64 queries by top-10, with score drift below 5×10^{-3}.

One detail from that exercise is worth recording because it took two failed runs to find. The reference must score against the **fp16 index that ships**, not against the fp32 vectors it was built from, and must not re-normalise after the cast. The two disagree on near-ties, and reporting the fp32 number would be reporting a gallery no deployment ever holds.

3. The anchor is load-bearing, and its absence is silent

3.1 Setup

A head fitted against one runtime does not transfer to another unchanged, because the two place their states in slightly different positions. The correction is a mean vector per modality: the deployment measures the mean of its own states over a few hundred anchor sentences and substitutes it for the one the head was trained with. Nothing is retrained.

We measure this on a runtime pair the companion paper did not test: a head fitted on PyTorch/fp16 states, scored through candle at Q4_0 — the arithmetic that actually runs in the browser. Same head, same gallery of 1,000 images, same 5,001 captions, different runtime.

Anchors are 200 captions drawn from training-split images, disjoint from the evaluation gallery. This matters: an earlier pass measured the anchor on the evaluation captions themselves and read R@1 0.235, which is transductive and inflated by roughly 0.04. The held-out number is the one below.

3.2 Result

runtime	t2i R@1	R@5	R@10
PyTorch fp16 (reference)	0.2300	0.4907	0.6215
candle Q4_0, head as-is	**0.0154**	0.0568	0.0896
candle Q4_0, + 4 KB anchor	**0.1952**	0.4321	0.5457

The uncorrected head loses a factor of fifteen. A single mean vector of 4,096 bytes, measured on 200 held-out sentences, recovers 85% of the reference. The residual −0.035 in R@1 is the quantizer's own cost and we report it rather than absorb it.

3.3 Why agreement metrics cannot see this

This is the part we consider the contribution rather than the number.

The natural way to validate that a head has ported correctly is to measure *agreement*: encode the same inputs on both runtimes, project both through the head, and compare the resulting head-space vectors or the rankings they induce. The companion paper reports 97.0% head-space retrieval agreement for a CUDA/bf16 to Apple-Silicon/MLX-Q4 transfer against a 99.96% same-runtime ceiling, and that measurement is sound.

But agreement applies the *same transform to both sides.* If the head's anchor is wrong for a runtime, it is wrong for every vector that runtime produces, in the same direction. The comparison is between two consistently displaced sets, and a consistent displacement is precisely what a comparison of that shape cannot detect. Both sides move together; the metric reads healthy.

The end task does not have this symmetry. The gallery was projected with the correct anchor and the query was not, so the displacement lands entirely in the gap between them, and R@1 falls off a cliff.

The failure is silent in the stronger sense too: the uncorrected system returns confidently ranked results with plausible scores. A human looking at the output sees search results. Only ground truth reveals that they are the wrong ones.

We therefore state a discipline. Portability of a read-out head may not be certified by any metric that applies the same transform to both sides of its comparison. It must be certified on a task with external ground truth.

3.4 The correction is per runtime, not per visitor

Every browser running this deployment executes the same quantized arithmetic, so the anchor is a constant across visitors. We measure it once and ship it as 4 KB rather than making each visitor spend a minute of compute re-deriving a constant. The deployment exposes a check — encode a handful of sentences locally, compare to the shipped anchor by cosine — so that a visitor whose runtime differs is told rather than silently served bad results.

4. Retrieval at deployment scale

Every browser-tier retrieval figure we had reported before this work used a 1,000-image evaluation pool. The deployed gallery has 123,287. That is 122,287 additional chances for something to outrank the right answer, and the difference is not a rounding correction.

Same captions, same ground truth, only the distractor pool grows:

head	pool	R@1	R@5	R@10	median rank
4K-pair head	1,000	0.2306	0.4903	0.6209	6
4K-pair head	123,287	0.0148	0.0406	0.0666	596
118K-pair head	1,000	0.4959	0.8166	0.8978	2
118K-pair head	123,287	0.0628	0.1568	0.2222	79
final head (§7)	123,287	**0.1108**	0.2510	0.3373	**33**

Two things follow.

Report the deployed pool. A fifteen-fold difference between the benchmark number and the shipped number is not a detail. We consider any retrieval figure quoted without its pool size to be uninterpretable, including our own earlier ones.

Median rank is the more honest summary at scale. R@1 0.1108 sounds weak. Median rank 33 out of 123,287 means the correct image typically lands in the top 0.027% — the read-out is doing substantial work that R@1 obscures. We report both.

We also note a measurement caveat that cuts against our own numbers being too *low*: with 123,287 photographs, many are equally good answers to a given caption, and the metric scores a *better* match than the gold image as a miss. We have not attempted to correct for this, and we do not claim the corrected figure would be dramatically different — only that the ceiling here is not 1.0 in any meaningful sense.

5. Steering the read-out, calibrated on retention

5.1 Mechanism

A direction in head space can be added to a projected query to shift what the gallery returns: `q' = normalise(q + α·axis)`. The axis is a difference of class means over already-projected vectors, so it costs one offline encode of two small sentence sets, and the artifact is roughly 2 KB. At query time the model is not involved at all — no hooks, no re-encoding.

We validate the mechanism in a form designed to fail if it were spurious. Axes are built from **captions** and applied to **image** queries, so a positive result is a claim about a shared space rather than about memorised neighbours, and the captions used to build an axis are held out from the images evaluated. Every point is measured against 32 matched-norm random axes.

5.2 The trap

Class purity — the fraction of retrieved images belonging to the target class — rises monotonically with α, all the way to 0.9. Optimising it is catastrophic, because past roughly $\alpha = 1$ the axis has *replaced* the query rather than steered it, and every query returns the same images regardless of what was asked.

We therefore measure **retention**: the share of the query's own unsteered top-k that survives steering. Retention near 1 means nothing happened; retention near 0 means the query no longer matters.

α	class purity (3 contrasts)	random control	retention
0	0.024 / 0.004 / 0.009	same	1.00
0.5	0.252 / 0.117 / 0.128	0.009–0.024	0.61–0.77
1.0	0.744 / 0.640 / 0.587	0.010–0.025	0.13–0.29
2.0	0.908 / 0.851 / 0.895	0.013–0.029	**0.01–0.03**

The matched random axes never leave baseline at any α, with z-scores for the real axis of 108 to 302 at the operating point. The direction carries meaning; it is not degrading the query into a class prior.

The high-α numbers are not the result. At $\alpha = 2$ retention is 0.01. We set the calibrated default at $\alpha = 0.5$ on retention, not on the best-looking purity, and we expose the retention measurement in the deployed interface so that a strength control cannot mislead the person operating it.

This is the second instance in this program of the same error shape: an earlier residual-stream steering result was initially read from likelihood on target-class text and gave an operating point roughly eight times past the one a behavioural measurement on neutral inputs supports.

5.3 A scope decision

Head-space query steering and residual-stream behavioural steering are different artifacts with different risk profiles, and presenting them under one word would misrepresent both. The deployed browser system exposes no hook into generation at all: the reply is produced by the unmodified model, and that boundary is structural rather than editorial.

6. What the payload buys

6.1 int8 index storage is free

At 123,287 images the binding constraint is resident memory, not download. An f32 index is 505 MB in RAM, which no phone will hold. Measured on real retrieval, with rows unit-norm so a single symmetric scale per row suffices:

store	t2i R@1	123K gallery
f32	0.2300	505 MB
f16	0.2300	252 MB
int8, per-row scale	**0.2306**	**127 MB**

int8 costs nothing measurable and quarters the memory. Python writes the format, Rust reads it, and top-1 agreement between int8 and f16 on the real fixture is 1.0000 over 64 queries.

6.2 Serving the images at all

A detail that generalises: `images.cocodataset.org`, the canonical host for COCO images, serves HTTP only. We verified this — HTTPS refuses the connection. Any page delivered over TLS therefore has every thumbnail blocked as mixed content, *silently*, while the retrieval scores remain perfectly correct. The visible symptom is a grid of broken frames that looks exactly like bad retrieval.

Our first mirror attempt uploaded 123,287 individual thumbnail files and moved 10,338 of them in three hours; per-file HTTP overhead dominates completely and the job would have taken 36 hours. Packing them into one 484 MB blob with a 1 MB offset table indexed by gallery row took seconds, and a browser fetches the twelve it needs with byte-range requests, which is what it wanted to do anyway.

7. Ablation: four of six predictions wrong

With the payload fixed, we asked what improves retrieval at 380 MB. Six levers were tested, and predictions were registered in the experiment script before the first arm ran.

arm	change	i2t R@1	Δ	shipped head
baseline	1 caption, last-token, linear	0.4236	—	2.1 MB
meanpool	mean over positions	**0.4668**	**+0.043**	2.1 MB
captions5	all 5 COCO captions	0.4552	+0.032	2.1 MB
layers3	concat L22+L25+L28	0.4192	−0.004	6.3 MB
epochs40	20 → 40 epochs	0.4112	−0.012	2.1 MB
imgmlp	non-linear image head	0.3154	**−0.108**	2.1 MB
all	everything at once	0.4636	+0.040	6.3 MB
meanpool + captions5, 40 ep		**0.5348**	**+0.111**	2.1 MB

Predicted: captions5 largest; layers3 modest positive; meanpool uncertain; imgmlp unknown but free; epochs40 small positive; combination better than any single lever but less than their sum.

Measured: meanpool was the largest single lever, not captions5. layers3 was null and cost 4 MB. epochs40 was negative. imgmlp was the worst arm by a wide margin. And the combination was *super-additive* — +0.043 and +0.032 alone, +0.111 together — not sub-additive.

Two methodological notes.

Bundling hid the answer. The `all` arm combined the two winners with three levers that hurt, and scored 0.4636 — *worse than mean pooling alone.* The winning pair was never tested in isolation until we noticed the gap in the design. An "everything" arm is not a substitute for testing the winners together.

A confound was real. More epochs hurt (−0.012) with one caption per image and helped (+0.010) with five. Training longer on a fixed set is just more overfitting; with five times the unique captions there is more to learn. Reporting "more epochs hurt" without that conditional would have been wrong.

The final configuration — mean pooling, all five captions sampled one per epoch so that five captions of one image never sit in a batch as false negatives, 40 epochs, single layer, linear image head — reaches t2i R@1 0.1108 on the full 123,287 gallery against 0.0148 for the original head, with **no change to the download**.

8. Negative results

Recorded because they constrain the design space, and because two of them contradict things we expected.

Free capacity on the image side is harmful. The image head ships no bytes, so a deeper one is free in payload terms. We predicted "unknown, and free" and treated it as the most exciting structural insight available. It was the worst intervention we tested (−0.108). Free in bytes is not free in statistics: a 25M-parameter MLP overfits 118K pairs where a 5M-parameter linear map does not. The asymmetry we identified is real; the conclusion we drew from it was wrong.

Multi-layer read-out is null here. The residual stream nests, so handing the head L22, L25 and L28 lets it use the deltas that a single-layer read-out cannot see, and the forward already computes them. Measured: +0.004, at a cost of 4 MB. Dropped.

The shared space does not represent word order. For 141 COCO captions we built the minimal pair that separates structure from vocabulary: the true caption against the same words with two noun phrases exchanged, *a woman walking a horse* against *a horse walking a woman*. Symmetric coordinations were excluded, because *a giraffe and a zebra* and *a zebra and a giraffe* describe the same photograph and would drive the score to chance by construction; 99 of 240 candidates went that way. Scored through the browser's own runtime, accuracy is **0.468** (95% CI 0.386 to 0.550, $z = -0.76$ against chance), with a mean margin of −0.004 in favour of the *swapped* caption. The read-out scores the words and not their arrangement.

This bounds what the correspondence represents and it costs none of the retrieval numbers, which never depended on word order. It is consistent with the literature that motivated SugarCrepe: contrastively trained dual encoders behave like bags of words on exactly this test, and a linear head on frozen backbones inherits that rather than repairing it. Anyone extending this work to tasks where binding matters, *the cat on the mat* against *the mat on the cat*, should treat this as the boundary it is.

Abliteration is not a lever for read-out. On the image side, a head trained and scored on an abliterated 27B tower reached i2t R@1 0.410 against 0.433 for base-on-base, and heads transferred across the base/abliterated boundary nearly intact (0.413, 0.418). The behavioural edit is roughly eighteen times smaller in displacement than switching quantizers. We also note that the tempting variant — projecting a refusal direction out of the state before the head — is theoretically dead for a *linear* head, which can already null any fixed direction; composing a projection with W yields another linear map that W could have learned.

The text-tower layer scan is censored. Read-out quality climbs monotonically over the last three probes (0.293 → 0.351 → 0.401) and L28 is the final block of the 0.6B model. The scan did not find an interior optimum; it ran out of model. We report this as an unresolved boundary rather than a selected layer.

9. Limitations

The text tower is the untested bottleneck. The censored layer scan points at tower depth as the binding constraint, and we did not test a larger tower because the deployment budget fixes the payload at 380 MB. Whether the curve continues past where the 0.6B model runs out of layers is open, and it is the single most informative experiment remaining.

One backbone pair, one domain. All deployment measurements use a Qwen3-0.6B text tower against a Qwen3.8-27B image tower on COCO. The companion paper's invariance evidence spans architectures; this paper's deployment evidence does not.

The anchor was validated on one runtime pair. We measured PyTorch/fp16 to candle/Q4_0. The silence argument in §3.3 is structural and we expect it to generalise, but we have measured it once.

Steering is validated on keyword-defined classes. The contrasts are crude by construction, chosen to be objective rather than subtle. Whether retention calibration transfers to finer semantic axes is untested.

Generation quality is not evaluated. We report that the model answers and that the answer is produced unmodified. We make no claim about how well a 0.6B model converses.

10. Reproduction

Every number in this paper is produced by a script in the public research repository and backed by a JSON artifact. The read-out geometry is a public Rust crate with no model dependency; the deployment runtime and browser engine are separate. Model heads, galleries, the packed thumbnail mirror, and the raw hidden states from which any head can be refitted are published.

Two reproduction disciplines are enforced in code rather than described in prose. Fixture-based parity tests fail if the Rust read-out drifts from the Python path that produced the published numbers, and an environment variable converts a missing fixture into a failure rather than a silent skip, because a skip that passes quietly is the same hazard the tests exist to catch. The gallery encoder refuses to run until it reproduces vectors already in the shipped gallery to a cosine above 0.99 — a pooling or layer mismatch produces vectors that look entirely reasonable on their own and are quietly incomparable with what they are meant to extend.

11. Conclusion

The companion result — that readable structure survives every axis a deployment can vary — is what makes a 0.6B model in a browser tab able to search a gallery a 27B model encoded. This paper is the other half of that sentence: surviving the journey is not the same as arriving aimed correctly.

The three failures we report share a shape. Each produces output that is confident, plausible, and wrong; each is invisible to the metric one would naturally reach for; and each is cheap to detect once you know to look. A 4 KB vector is the difference between R@1 0.0154 and 0.195. A pool size is the difference between 0.2306 and 0.0148. A retention measurement is the difference between steering a query and deleting it.

We think the general lesson is that read-out deployment needs its own measurement discipline, distinct from the one that establishes the read-out exists. The instrument being correct in principle, and the instrument being pointed at the thing on the visitor's device, are different claims requiring different evidence.

References

Lancaster, J. B. (2026). *Train Once, Read Everywhere: Substrate Invariance of the Linearly Readable Structure in Frozen Language Models.* SSRN Working Paper No. 7264778. https://papers.ssrn.com/sol3/papers.cfm?abstract_id=7264778

Chen, X., Fang, H., Lin, T.-Y., Vedantam, R., Gupta, S., Dollár, P., & Zitnick, C. L. (2015). *Microsoft COCO Captions: Data Collection and Evaluation Server.* arXiv:1504.00325.

Ethayarajh, K. (2019). *How Contextual are Contextualized Word Representations? Comparing the Geometry of BERT, ELMo, and GPT-2 Embeddings.* EMNLP 2019. (Source of the anisotropy correction applied to every cosine reported here.)

Faghri, F., Fleet, D. J., Kiros, J. R., & Fidler, S. (2018). *VSE++: Improving Visual-Semantic Embeddings with Hard Negatives.* BMVC 2018.

Hsieh, C.-Y., Zhang, J., Ma, Z., Kembhavi, A., & Krishna, R. (2023). *SugarCrepe: Fixing Hackable Benchmarks for Vision-Language Compositionality.* NeurIPS 2023 Datasets and Benchmarks. (Source of the parity replay fixture.)

Radford, A., Kim, J. W., Hallacy, C., et al. (2021). *Learning Transferable Visual Models From Natural Language Supervision.* ICML 2021.

Qwen Team (2025). *Qwen3 Technical Report.* arXiv:2505.09388.

`huggingface/candle` — Minimalist ML framework for Rust. MIT/Apache-2.0. The browser tier vendors its quantized Qwen3 implementation, modified only to expose a hidden-state tap.